Sorting in assembly

DM548
Computer Architecture
(& System Programming)
Syddansk Universitet

Martin Kuntz Jacobsen: marja23

2024-11-03

Table of contents

1 Introduction **3**

2 Design **3**
2.1 Maximizing sorting speed 3
2.2 Minimizing binary size 4

3 Implementation **5**
3.1 General 5
3.2 Maximizing sorting speed 5
3.3 Minimizing binary size 7

4 Benchmark **9**

5 Discussion **10**

6 Conclusion **10**

1 Introduction

Sorting is an often encountered program in computer science, often necessitating the need for fast or small sorting algorithms depending on the specific problem. This project explores the implementation of two distinct sorting programs in assembly x86-64, each with its own goal: one optimized for the fastest sorting speed and the other for the smallest binary size. This project specifically focuses on sorting (x, y)-coordinates based on their y coordinate.

The data for this project is a text file with multiple lines, where each line has the format $\langle x \rangle \backslash t \langle y \rangle \backslash n$. here, both $\langle x \rangle$ and $\langle y \rangle$ are integers in the range 0 to 32767, both inclusive. It is not required for the sorting algorithm to maintain the original order of elements, making the sorting algorithm stable.

This project explores the trade-offs between speed and size in low-level programming through the development of these two programs. By analyzing each implementation it gives a better understanding of how algorithms influence the efficiency.

2 Design

2.1 Maximizing sorting speed

The decision that has one of the biggest impacts on performance is what sorting algorithm to implement. Due to the project's small input size Counting Sort was chosen because of its time complexity O(n+k) and the algorithm in general being fast despite the lag of cache efficiency.

- Quick Sort: This is another fast sorting algorithm with an average case efficiency of O(n log n). It works in place and has good cache efficiency because of the repeated sequential iteration. It is possible to use Quick Sort even though it is not stable by storing x and y in 32 bits and only sorting based on the 16 y bits.
- Merge Sort: This is one of the most used algorithms because it is stable, has good caching and runs in O(n log n). Its downside is that it has a lot of overhead for small lists and is not quite as fast as Quick Sort on random data.
- Insertion Sort: It is one of the simplest sorting algorithms to implement and does not scale great because of its time complexity of $O(n^2)$.

There exist many more sorting algorithms each with their specific trade-off, these are some of the most common or/and most suited algorithms when considering sorting speed. The choice of Counting Sort is particularly suited to this problem since the input range is defined. (See Algorithm 1)

Merge Sort could most likely have allowed for the same speed or better if threading was used, as this would reduce the latency of IO operations by reading the file concurrently. This could even have been combined with Counting Sort or Quick Sort. It was however deliberately chosen not to use threading to maintain simplicity and instead focus on the sorting algorithm itself and what other optimizations can improve program performance.

Algorithm 1: Counting Sort slightly modified

Input: An array A with n elements each an integer in the range 0 to k
Output: Sorted array B

```
// Zero out counting array
```
C[0:k] $\leftarrow 0$;
```
// Count occurrences
```
for *each* i *from* 0 *to* $n-1$ **do**
    $C[A[i]] += 1$;
```
// Cumulative count
```
$s \leftarrow 0$;
for *each* i *from* 0 *to* k **do**
    $s += C[i]$;
    $C[i] \leftarrow s$;
```
// Sort array
```
for *each* i *from* 0 *to* $n-1$ **do**
    $C[A[i]] \leftarrow C[A[i]] - 1$;
    $B[C[A[i]]] \leftarrow A[i]$;

2.2 Minimizing binary size

To make the smallest binary possible for sorting a file, the biggest consideration is how to use the least logic to sort a file. For this one can use that the input range is defined as 0-32767. This allows just iterating the file 32767 times and printing to the console if $\langle y \rangle$ is that number. This allows skipping proper parsing of the file since we can just parse $\langle y \rangle$ each time and print the line if $\langle y \rangle$ is the count. This also avoids the conversion from numbers back into a string. (See Algorithm 2)

Algorithm 2: Force Sort

Input: A file containing pairs (x, y) where $0 \leq y \leq n$
Output: Nothing

for *each* i *from* 0 *to* n **do**
    for *each line in file* **do**
        $y \leftarrow$ extract_y($line$) ;
        if $y == i$ **then**
            print(line) ;

3 Implementation

3.1 General

For both implementations, some similar optimizations have been used. The first optimization is to ignore any errors that can occur after syscalls. This is not the best way to handle syscalls, but allows for quicker execution and smaller binaries. Both programs assume the input data is correct and a file is specified. The programs still follow the Linux calling convention meaning no assumptions are made on non-callee saved registers after syscalls.

Both implementations also use a lot of optimization in the choice of instructions. One of these optimizations is using **xor** to set a register to 0. This both saves multiple bytes in the binary and is also faster on modern architecture.[1] (See Listing 1)

```
1    movl $0, %eax        # unoptimized
2    xorl %eax, %eax      # optimized
```

Listing 1: Clear register

The next optimization is using **test** which does an **and** and discards the result. This allows for checking if a register is 0 and is smaller in the binary. (See Listing 2)

```
1    cmpl $0, %eax        # unoptimized
2    jz   _label
3    testl %eax, %eax     # optimized
4    jz   _label
```

Listing 2: Test if register is 0

The register choice is also optimized for both programs meaning 32 bit registers for **eax**, **ecx**, **edx**, **ebx**, **esi**, **edi**, **esp** and **ebp** are used whenever possible as it saves one byte because it doesn't have the 64 bit prefix. Using 32 bit registers also clears the upper 32 bits meaning **mov $5, %eax** is identical to **mov $5, %rax** while saving 1 byte. The usage of **sub** or **add** is often also preferred over **cmp** as it is often possible to use the result after.

3.2 Maximizing sorting speed

The program is made such that the maximum number of coordinates the program can process is 2^{32}. Although the problem is a sorting problem it kinda isn't when the correct algorithm is chosen for sorting. The program only uses 10% of the time in user space and 5% of the time in kernel space. This means that most of the time is spent waiting for IO operations this therefore makes it the most important part to optimize.

There exist multiple different ways to process a file, one way is to just go through the file character by character. This however leads to a lot of time being spent waiting for IO. The next thing one might try is to use buffered reading of the file. This is certainly an improvement, but we can do better than that. Instead of handling reading the file and fetching each buffer, we can instead look to **mmap**[2] which makes the OS do all the heavy lifting of reading the file into the virtual

[1] https://stackoverflow.com/questions/17981447/microarchitectural-zeroing-of-a-register-via-the-register-renamer-performance-v/18027854#18027854

[2] https://manpages.debian.org/unstable/manpages-dev/mmap.2.en.html

memory of the process. Using **mmap** is faster when working with larger files than 10MB.[3] The program is optimized for reading files 5.000.000 (x,y)-coordinates, which file size can be estimated by multiplying the number of lines by 10 resulting 50MB.

Now we know how to read the file efficiently but why not tell the OS as well. This is where **madvice** [4] comes in. This allows the program to announce how it plans on using a given memory space, here MADV_SEQUENTIAL is used. This allows the OS to aggressively read ahead and free soon after access.[5]

Now that the data is lazy loaded into virtual memory we can just read directly from it. But before reading a memory allocation is made using **brk**[6] for entire size of the file. This is done because it later on allows converting the sorted array to a single string in memory and then calling write. The program starts by using this buffer to store the parsed array. (see Figure 1) It is not possible to know the size of the array before parsing therefore it doesn't allocate space for the sorted array.

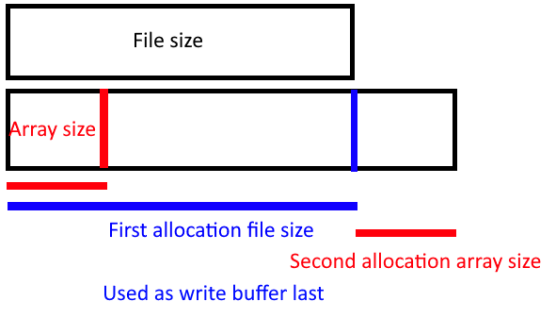


Figure 1: **brk** memory allocations

The naive way to parse the strings into numbers would be to use the **imul** instruction which allows multiplying 2 numbers and then doing an addition for each character. (see Listing 3) This could end up taking up at least 4 clock cycles on most computers, but luckily there is an optimized way to multiply by some constants.[7] This can be done by abusing **lea** to do arithmetic as **lea** computes $base + index * scale + displacement$. (see Listing 4) This takes only 2 clock cycles.

```
1    imulq $10, %rcx        # rcx = rcx * 10
2    addq  %rax, %rcx       # rcx = rcx + rax
```

Listing 3: Unoptimized multiplication by constant and addition

```
1    leaq (%rcx, %rcx, 4), %rcx    # rcx = rcx * 5
2    leaq (%rax, %rcx, 2), %rcx    # rcx = rcx * 2 + rax
```

Listing 4: Optimized multiplication by constant and addition

[3] https://github.com/david-slatinek/c-read-vs.-mmap/
[4] https://manpages.debian.org/unstable/manpages-dev/madvise.2.en.html
[5] https://manpages.debian.org/unstable/manpages-dev/madvise.2.en.html
[6] https://manpages.debian.org/unstable/manpages-dev/brk.2.en.html
[7] https://www.agner.org/optimize/instruction_tables.pdf

The (x,y)-coordinates are stored as tuples of words in an array starting at the previous **brk** end address. Another thing the program does while parsing the data is to count $\langle y \rangle$ in the counting array. This avoids an entire loop, but it does however introduce a bit more branching which usually one should avoid.

The counting array is placed in the BSS segment as **.space** of size $4 * 32768$. This is done because the BSS segment is automatically initialized as zero by default. This therefore avoids dynamic allocation and clearing.

After parsing the file the next step in Counting Sort is to accumulate the number of integers at each index and replace the value in the counting array. This also gives the entire length of the array which makes it possible to allocate the final memory needed as seen in Figure 1. This is once again done using **brk**[8] by moving the end address $sum * 4$.

Lastly, there is the conversion from an array back into a string that can be written to stdout. This is done by having a lookup table with each number the amount of digits and the number as a string padded to fill 8 bytes. (see Listing 5) This makes it possible to skip what in most cases would be 5 divisions which each taking 10-20 cycles. Each of these divisions also depends on each other making it even slower. Another method to divide by a constant is called division by invariant multiplication which could also serve as a performant alternative, however, this option has not been considered. The hope is that the lookup table will end up in the L2 or L3 cache because of the frequent accesses $(filelength * 2)$ thereby only taking 10-40 cycles. The string in the lookup table is then copied into the buffer using the **rep movsb** instruction.

```
1    .byte 1, '7','-','-','-','-','-','-'
2    .byte 1, '8','-','-','-','-','-','-'
3    .byte 1, '9','-','-','-','-','-','-'
4    .byte 2, '1','0','-','-','-','-','-'
5    .byte 2, '1','1','-','-','-','-','-'
6    .byte 2, '1','2','-','-','-','-','-'
```

Listing 5: Section of lookup table

Then the entire buffer with the same size as the file is written to stdout. Then the last minor optimization is applied which is not to close the file descriptor. This is not advised but allows the application to exit quicker with the kernel cleaning up the file descriptor as part of process termination.

3.3 Minimizing binary size

To make the sorting as small as possible every byte must be considered. When considering every byte, one of the worst used instructions is the immediate to register move instruction 32 bit (**movl**). This is because it takes 4 bytes for the immediate and 1 byte for the register and instruction. Instead for immediate values up to 255, **xorl %eax, %eax** and **xorb $1, %al** can be used to save one byte. If the 56 upper bits are already cleared or don't need to be cleared the clearing of the register can be avoided, now saving 3 bytes. This kind of optimization is what makes the program

[8] https://manpages.debian.org/unstable/manpages-dev/brk.2.en.html

binary so small at only 108 bytes without ELF format.

The program iterates each byte directly from read() without any buffering, as this avoids having any extra logic to iterate each chunk and fetch each chunk. When reading each line the program parses both x and y, but discards x after parsing as this allowed for less additional code, reducing the binary size. The program opens and closes the file each of the 32.768 iterations as this avoids having to use the syscall **lseek**[9]. The reason for avoiding **lseek** is that it requires setting up 2 more registers than **close**[10]. **lseek** is a direct alternative to **close** as it would allow not closing the file. The reason why closing the file is required is that Linux has a file handle soft limit of 1.024 which is less than the would be needed 32.768 file handles.[11]

The outer iteration loop can also be done in a more optimal way than usual, because of the limited input size 0 to $2^{15} - 1$. (see Listing 6 and Listing 7)

```
    xorl %eax, %eax
_loop:
    # CODE
    incl %eax
    cmpl $32768, %eax
    jne   _loop
```

Listing 6: Unoptimized for each loop size 32.768

```
    xorl %eax, %eax
_loop:
    # CODE
    incw %ax
    jno  _loop                  # used that 32.768 overflows in word size
```

Listing 7: Optimized for each loop size 32.768

With these optimizations and many more the program code only takes up 108 bytes, but when the program is assembled it has the binary size of 4.896 bytes. What is going on here, where did all the extra bytes come from? This is the result of paging and additional bundled information which can be disabled using the **ld** linker arguments **–discard-all –relax –strip-all**. Now when the program is assembled it results in a file size of 370 + 108. This still seems quite large being 4 times the size of the actual program. The 370 additional bytes come from the ELF[12] format. Although all this data might seem necessary it includes section data even though the program has no section data. To reduce the program size further the **ld** linker argument **–oformat binary** can be used to assemble the program without the additional ELF format. This now means we have to directly specify the header in program code. This is done by inserting the required information found in the wiki[13]. The minimized ELF format includes an ELF header and a program header but avoids the unnecessary section headers that usually are included.

[9] https://manpages.debian.org/unstable/manpages-dev/lseek.2.en.html
[10] https://manpages.debian.org/unstable/manpages-dev/close.2.en.html
[11] https://www.baeldung.com/linux/limit-file-descriptors
[12] https://en.wikipedia.org/wiki/Executable_and_Linkable_Format
[13] https://en.wikipedia.org/wiki/Executable_and_Linkable_Format

4 Benchmark

The asymptotic runtime of both Counting Sort and Force Sort is O(n+k), which falls right in line with the benchmark run for both programs. (see Table 1) For the Counting Sort program, practically no difference can be noticed up till input size 10.000. This is likely because of the varying time it takes to read the file and the program startup time. After 10.000 the algorithm scales linearly as expected. The thing that is not visible in the table is how the program drastically improves with warm start instead of cold start. (see Listing 8) This is however most prevalent in files with over 1 million (x,y)-coordinates and caps out at a two times improvement.

The runtime for the Force Sort program seems to improve as the input file gets bigger. This makes a lot of sense since it opens and closes the file 32.768 which one could expect to have quite some overhead. The insanely long runtimes also make sense since it parses the data 32.768 times and reads each character from the file one at a time 32.768 times.

Running the time command on the Counting Sort program with a file with 5 million (x,y)-coordinates results in quite different values with 22 seconds elapsed, 2 seconds user and 0.8 seconds system. What this means is most of the time the program runs is not even spent doing anything actively, but instead just waiting for data to be ready. This also matches what is seen when removing all sorting logic and parsing logic as the runtime barely changes.

The story is more or less the same for the Force Sort program with a file with 10 (x,y)-coordinates results in around 8 minutes elapsed, 2 seconds user and 20 seconds system. This also makes sense since the program is reading each character in the file 32.768 times and one at a time. The time spent in system therefore also makes a lot of sense as it just requires making a lot of syscalls.

Both of these sorting algorithms likely would not be useful in practical appliances. The Counting Sort program has the issue that it uses way more memory for only small performance gains. The small sorting algorithm has a very low memory footprint, but it has so bad performance as it is not even viable to run for even 1 input. The Counting Sort algorithm could be used in some cases where the entire unparsed file can fit into memory and performance is critical.

```
Run 1: 33.001729 seconds
Run 2: 28.570113 seconds
Run 3: 22.380085 seconds
Run 4: 23.290361 seconds
Run 5: 17.531864 seconds
```

Listing 8: Consequtive runtimes on 40.000.000 (x,y)-coordinates

Input size	1	10	100	1k	10k	100k	1m	5m	10m	20m	40m
Counting Sort	0.021s	0.017s	0.015s	0.019s	0.025s	0.074s	0.477s	2.432s	4.914s	10.788s	20.630s
Force Sort	81.332s	499.090s	4691.850s	-	-	-	-	-	-	-	-

Table 1: Performance of algorithms - avg over 10 runs

5 Discussion

The goal of the project was to create two separate sorting algorithms in x86-64 assembly language, each with its own goal: one optimized for the fastest sorting speed and the other for the smallest binary size.

For the fast sorting program, Counting Sort was implemented because of the limited input range from 0 to 32767. This allowed for sorting in O(n+k) time complexity, which becomes very efficient on larger datasets. The benchmarks also confirmed that the implementation runtime scaled linearly, while only taking up 10% of execution time. Any further real improvements would require the use of threading as most of the program's time is spent waiting on data.

The program focused entirely on reducing the binary size used a more streamlined algorithm, Force Sort. This algorithm was designed for the limited input size where it is possible to iterate all numbers. Eliminating the usage of large instructions and reconsidering every instruction allowed for the creation of a very small binary.

Both algorithms and implementations provide insight into how to optimize assembly code for different goals. The Counting Sort implementation could be viable in some applications where memory is not an issue, performance is critical and threading is not possible. Force Sort could be made viable for smaller files if memory is very limited and it is altered to only parse the file once. However, the implementation provided is never viable.

6 Conclusion

The implementations of Counting Sort for execution speed and Force Sort for binary size proved to be ideal for each of their respective benchmarks. Both algorithms were able to utilize the limited input range making them better than would otherwise be possible. Counting sort ended up being very ideal with it taking up less than 10% of the execution time. Force Sort was ideal for minimizing binary size with the final binary only taking up 228 bytes.

Overall, the project shows how optimizing for only a single benchmark results in much worse in other areas as the last minuscule improvements require considerable trade-offs. Both the Counting Sort implementation and Force Sort implementation could be made more viable in real applications by considering other metrics just a tiny bit without huge deficits in the focused benchmark.